Enable internet access for VPC-connected Lambda functions

By default, Lambda functions run in a Lambda-managed VPC that has internet access. To access resources in a VPC in your account, you can add a VPC configuration to a function. This restricts the function to resources within that VPC, unless the VPC has internet access. This page explains how to provide internet access to VPC-connected Lambda functions.

I don't have a VPC yet

Create the VPC

The **Create VPC workflow** creates all VPC resources required for a Lambda function to access the public internet from a private subnet, including subnets, NAT gateway, internet gateway, and route table entries.

To create the VPC

1. Open the Amazon VPC console at https://console.aws.amazon.com/vpc/.
2. On the dashboard, choose **Create VPC**.
3. For **Resources to create**, choose **VPC and more**.
4. **Configure the VPC**
   a. For **Name tag auto-generation**, enter a name for the VPC.
   b. For **IPv4 CIDR block**, you can keep the default suggestion, or alternatively you can enter the CIDR block required by your application or network.
   c. If your application communicates by using IPv6 addresses, choose **IPv6 CIDR block**, **Amazon-provided IPv6 CIDR block**.
5. **Configure the subnets**
   a. For **Number of Availability Zones**, choose **2**. We recommend at least two AZs for high availability.
   b. For **Number of public subnets**, choose **2**.
   c. For **Number of private subnets**, choose **2**.
   d. You can keep the default CIDR block for the public subnet, or alternatively you can expand **Customize subnet CIDR blocks** and enter a CIDR block. For more information, see Subnet CIDR blocks .
6. For **NAT gateways**, choose **1 per AZ** to improve resiliency.

7. For **Egress only internet gateway**, choose **Yes** if you opted to include an IPv6 CIDR block.
8. For **VPC endpoints**, keep the default (**S3 Gateway**). There is no cost for this option. For more information, see Types of VPC endpoints for Amazon S3.
9. For **DNS options**, keep the default settings.
10. Choose **Create VPC**.

Configure the Lambda function

To configure a VPC when you create a function

1. Open the Functions page of the Lambda console.
2. Choose **Create function**.
3. Under **Basic information**, for **Function name**, enter a name for your function.
4. Expand **Advanced settings**.
5. Select **Enable VPC**, and then choose a VPC.
6. (Optional) To allow outbound IPv6 traffic, select **Allow IPv6 traffic for dual-stack subnets**.
7. For **Subnets**, select all private subnets. The private subnets can access the internet through the NAT gateway. Connecting a function to a public subnet doesn't give it internet access.

> **Note**
>
> If you selected **Allow IPv6 traffic for dual-stack subnets**, all selected subnets must have an IPv4 CIDR block and an IPv6 CIDR block.

8. For **Security groups**, select a security group that allows outbound traffic.
9. Choose **Create function**.

Lambda automatically creates an execution role with the AWSLambdaVPCAccessExecutionRole AWS managed policy. The permissions in this policy are required only to create elastic network interfaces for the VPC configuration, not to invoke your function. To apply least-privilege permissions, you can remove the **AWSLambdaVPCAccessExecutionRole** policy from your execution role after you create the function and VPC configuration. For more information, see Required IAM permissions.

To configure a VPC for an existing function

To add a VPC configuration to an existing function, the function's execution role must have permission to create and manage elastic network interfaces. The AWSLambdaVPCAccessExecutionRole AWS managed policy includes the required permissions. To apply least-privilege permissions, you can remove the **AWSLambdaVPCAccessExecutionRole** policy from your execution role after you create the VPC configuration.

1. Open the Functions page of the Lambda console.
2. Choose a function.
3. Choose the **Configuration** tab, and then choose **VPC**.
4. Under **VPC**, choose **Edit**.
5. Select the VPC.
6. (Optional) To allow outbound IPv6 traffic, select **Allow IPv6 traffic for dual-stack subnets**.
7. For **Subnets**, select all private subnets. The private subnets can access the internet through the NAT gateway. Connecting a function to a public subnet doesn't give it internet access.

   > **Note**
   >
   > If you selected **Allow IPv6 traffic for dual-stack subnets**, all selected subnets must have an IPv4 CIDR block and an IPv6 CIDR block.

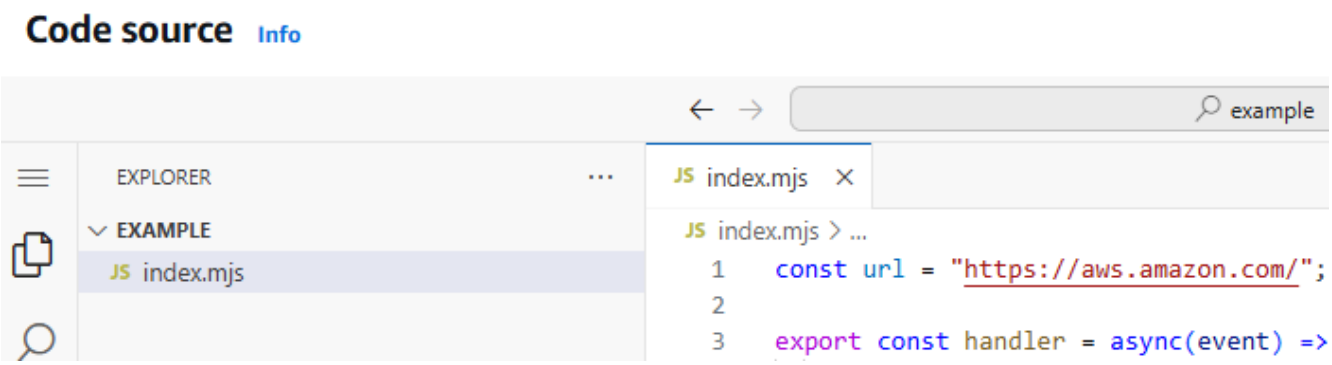
8. For **Security groups**, select a security group that allows outbound traffic.
9. Choose **Save**.

Test the function

Use the following sample code to confirm that your VPC-connected function can reach the public internet. If successful, the code returns a `200` status code. If unsuccessful, the function times out.

Node.js

1. In the **Code source** pane on the Lambda console, paste the following code into the **index.mjs** file. The function makes an HTTP GET request to a public endpoint and returns the HTTP response code to test if the function has access to the public internet.

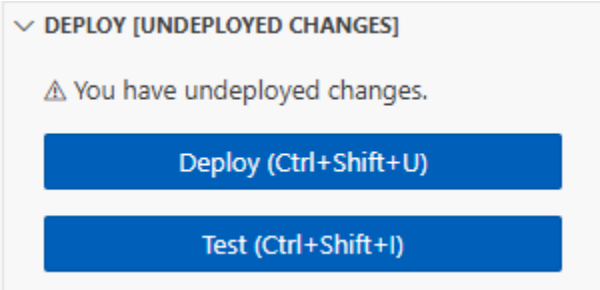


Example— HTTP request with async/await

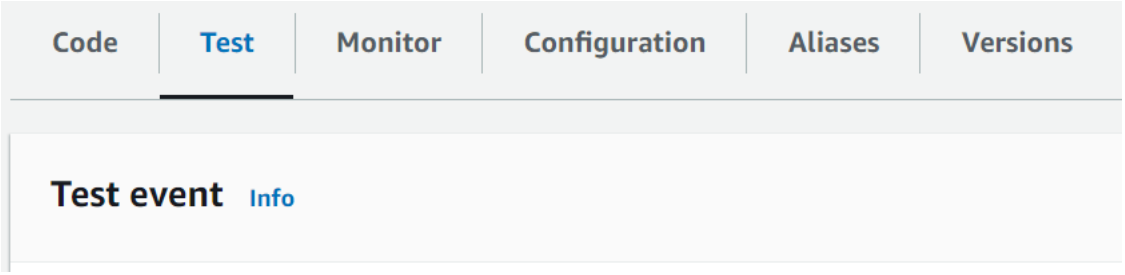
```
const url = "https://aws.amazon.com/";

export const handler = async(event) => {
    try {
        const res = await fetch(url);
        console.info("status", res.status);
        return res.status;
    }
    catch (e) {
        console.error(e);
        return 500;
    }
};
```

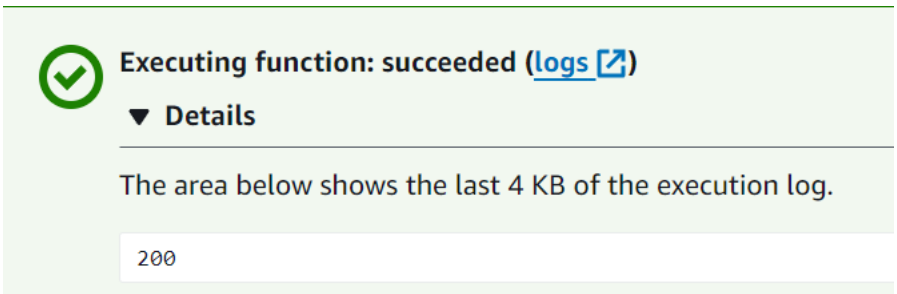
2. In the **DEPLOY** section, choose **Deploy** to update your function's code:



3. Choose the **Test** tab.



4. Choose **Test**.
5. The function returns a `200` status code. This means that the function has outbound internet access.

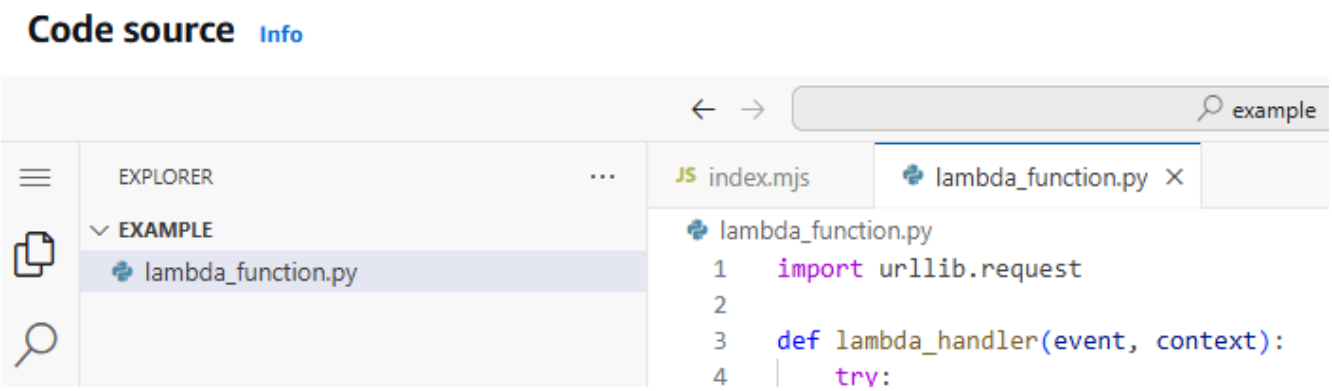


If the function can't reach the public internet, you get an error message like this:

```
{
  "errorMessage": "2024-04-11T17:22:20.857Z abe12jlc-640a-8157-0249-9be825c2y110
 Task timed out after 3.01 seconds"
}
```

Python

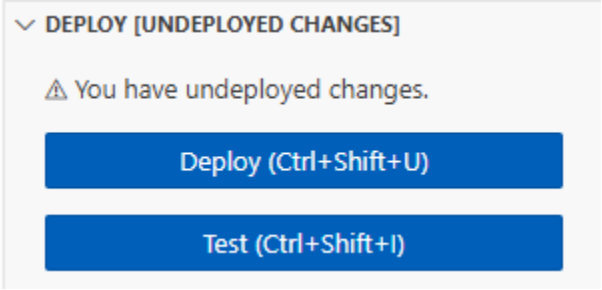
1. In the **Code source** pane on the Lambda console, paste the following code into the **lambda_function.py** file. The function makes an HTTP GET request to a public endpoint and returns the HTTP response code to test if the function has access to the public internet.



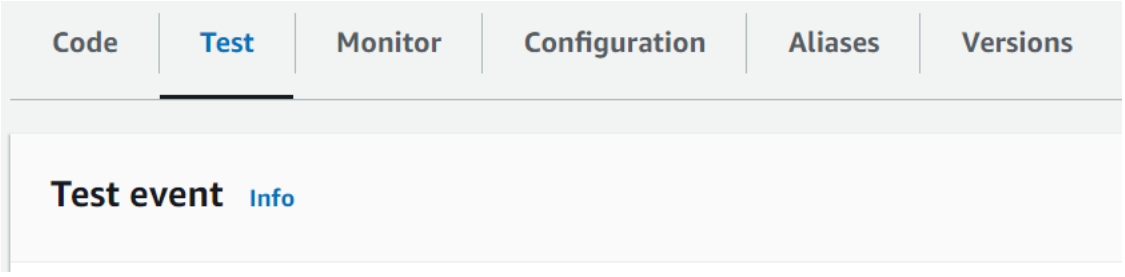

```
import urllib.request

def lambda_handler(event, context):
    try:
        response = urllib.request.urlopen('https://aws.amazon.com')
        status_code = response.getcode()
        print('Response Code:', status_code)
        return status_code
    except Exception as e:
        print('Error:', e)
        raise e
```

2. In the **DEPLOY** section, choose **Deploy** to update your function's code:

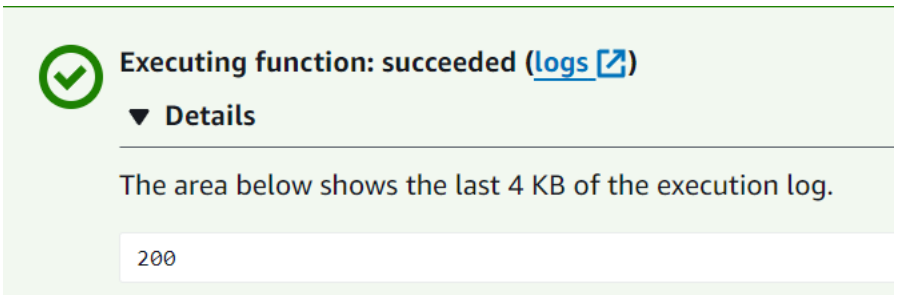


3. Choose the **Test** tab.



4. Choose **Test**.

5. The function returns a 200 status code. This means that the function has outbound internet access.



If the function can't reach the public internet, you get an error message like this:

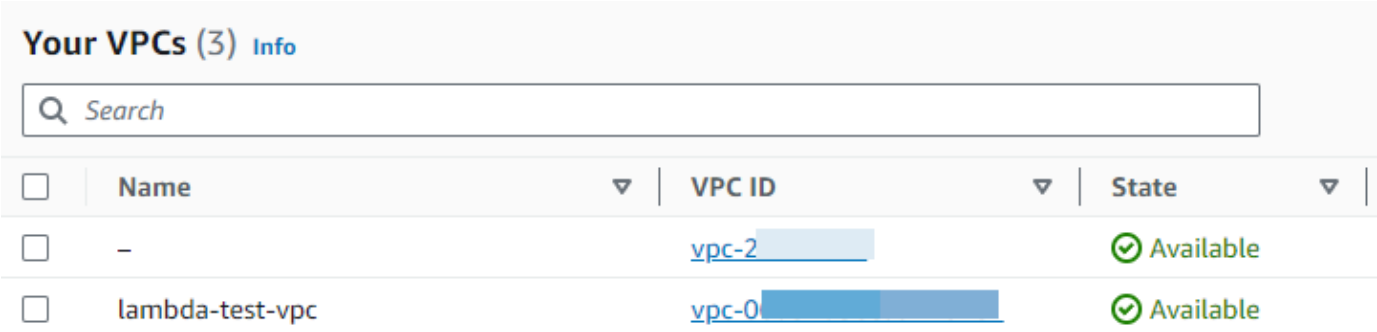
```
{
  "errorMessage": "2024-04-11T17:22:20.857Z abe12jlc-640a-8157-0249-9be825c2y110
 Task timed out after 3.01 seconds"
}
```

I already have a VPC

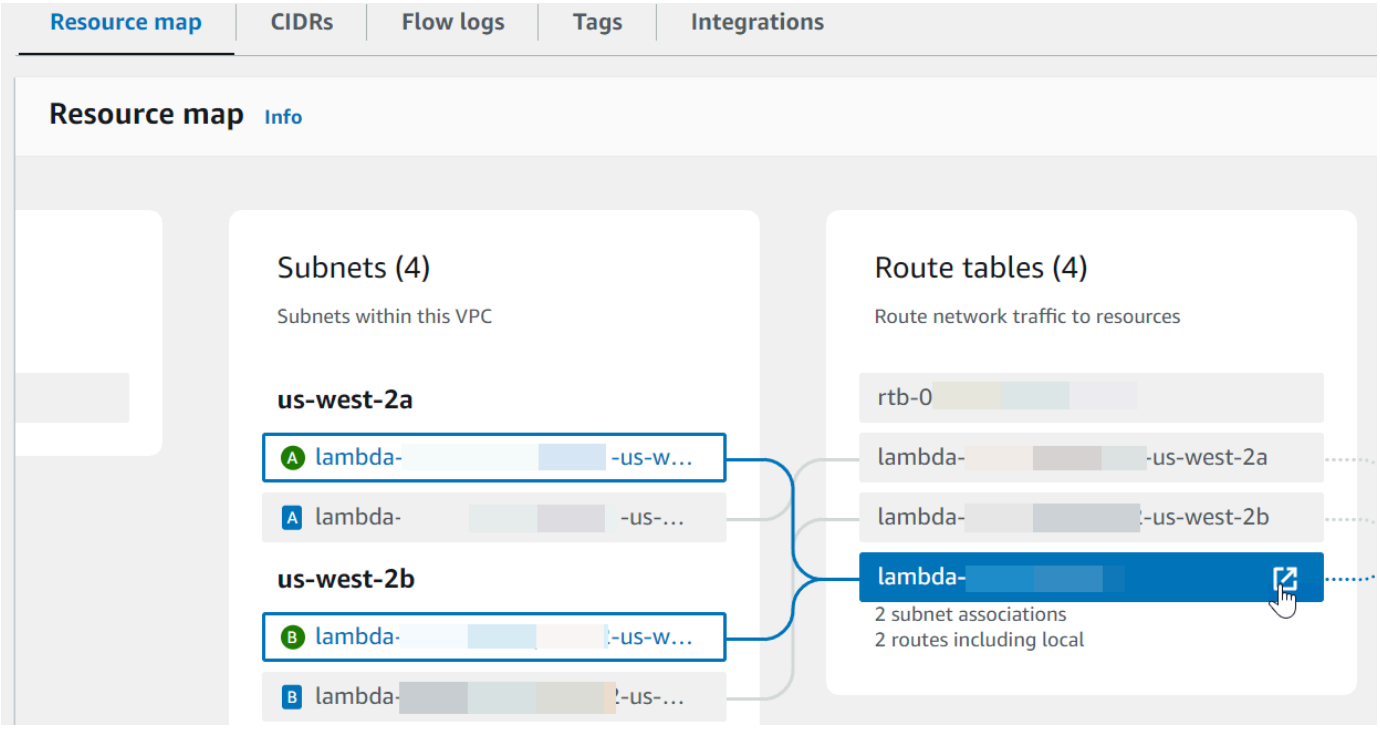
If you already have a VPC but you need to configure public internet access for a Lambda function, follow these steps. This procedure assumes that your VPC has at least two subnets. If you don't have two subnets, see Create a subnet in the *Amazon VPC User Guide*.

Verify the route table configuration

1. Open the Amazon VPC console at https://console.aws.amazon.com/vpc/.
2. Choose the **VPC ID**.



3. Scroll down to the **Resource map** section. Note the route table mappings. Open each route table that is mapped to a subnet.

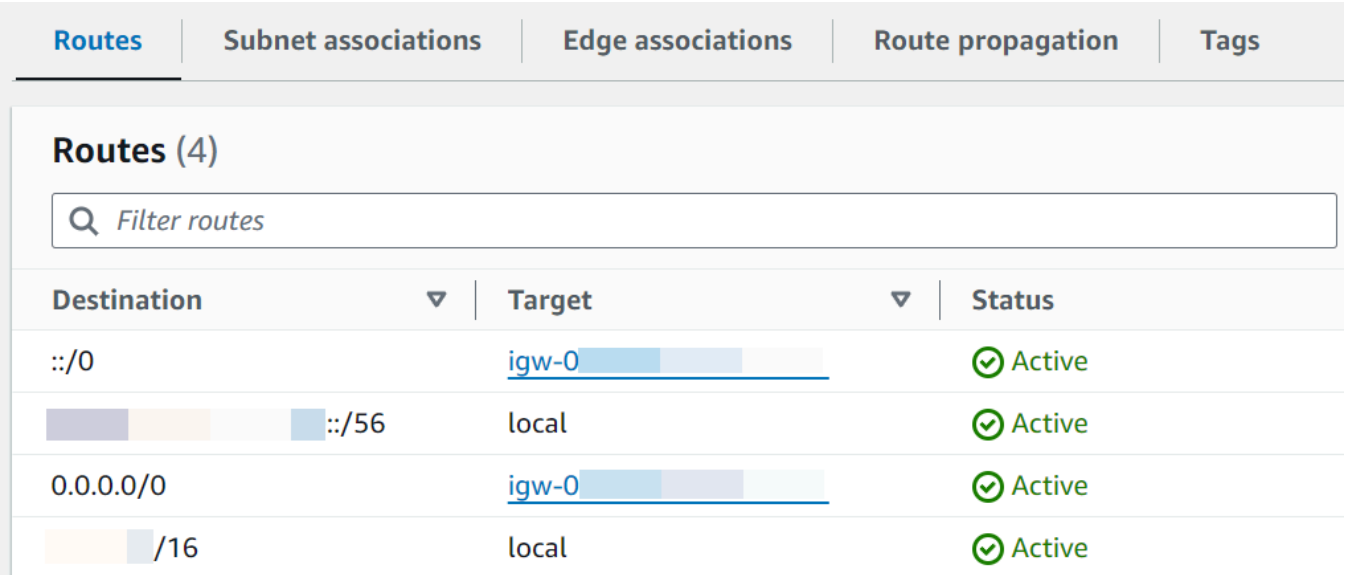


4. Scroll down to the **Routes** tab. Review the routes to determine if your VPC has both of the following route tables. Each of these requirements must be satisfied by a separate route table.

   - Internet-bound traffic (`0.0.0.0/0` for IPv4, `::/0` for IPv6) is routed to an internet gateway (`igw-xxxxxxxxxx`). This means that the subnet associated with the route table is a public subnet.

     > **Note**
     >
     > If your subnet doesn't have an IPv6 CIDR block, you will only see the IPv4 route (`0.0.0.0/0`).

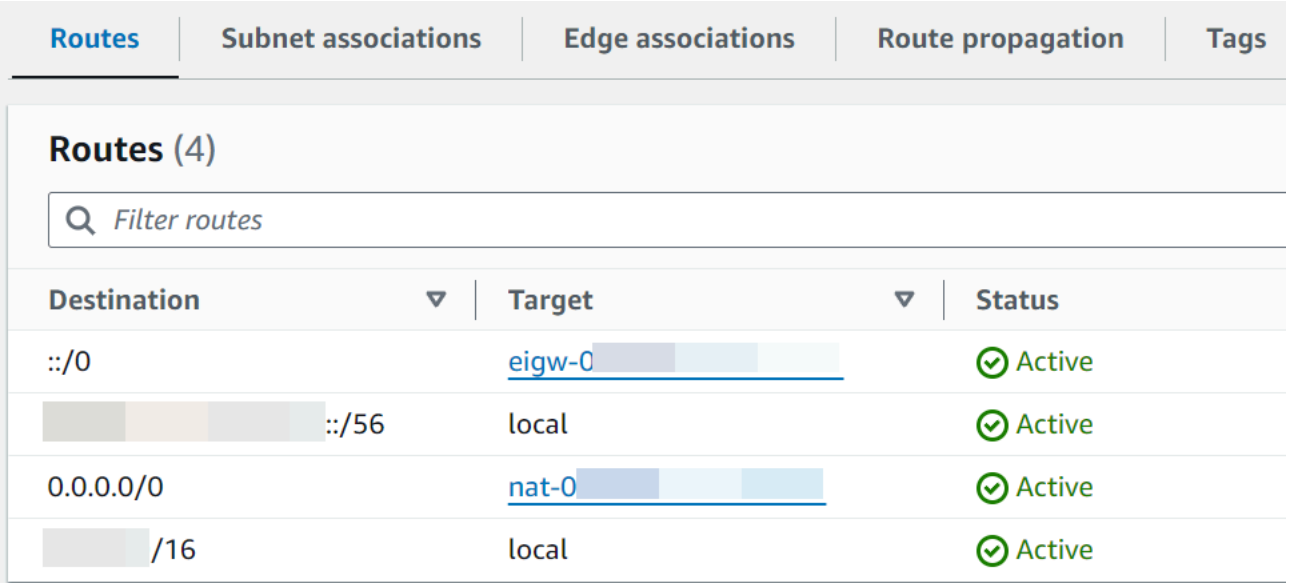
Example public subnet route table



- Internet-bound traffic for IPv4 (`0.0.0.0/0`) is routed to a NAT gateway (`nat-xxxxxxxxxx`) that is associated with a public subnet. This means that the subnet is a private subnet that can access the internet through the NAT gateway.

> **Note**
>
> If your subnet has an IPv6 CIDR block, the route table must also route internet-bound IPv6 traffic (`::/0`) to an egress-only internet gateway (`eigw-xxxxxxxxxx`). If your subnet doesn't have an IPv6 CIDR block, you will only see the IPv4 route (`0.0.0.0/0`).

Example private subnet route table



5. Repeat the previous step until you have reviewed each route table associated with a subnet in your VPC and confirmed that you have a route table with an internet gateway and a route table with a NAT gateway.

   If you don't have two route tables, one with a route to an internet gateway and one with a route to a NAT gateway, follow these steps to create the missing resources and route table entries.

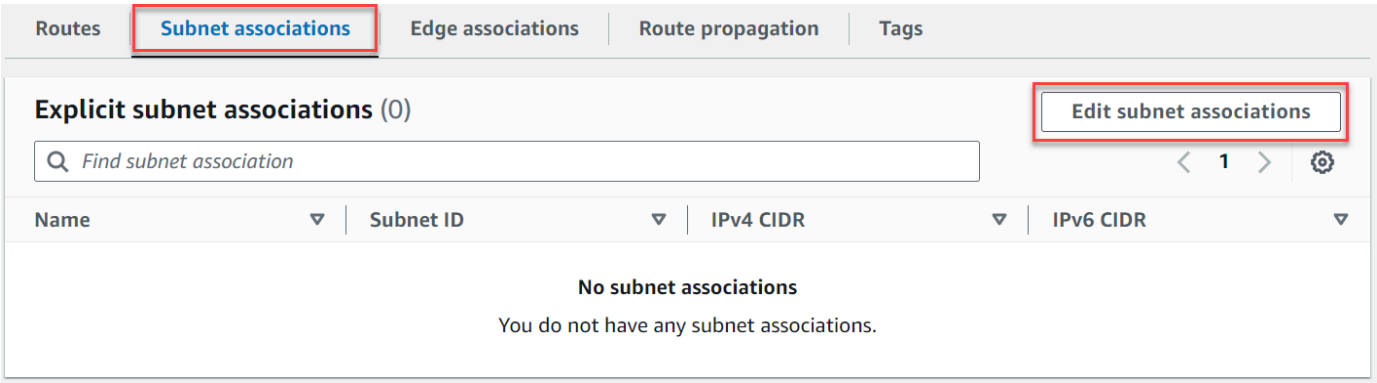
Create a route table

Follow these steps to create a route table and associate it with a subnet.

To create a custom route table using the Amazon VPC console

1. Open the Amazon VPC console at https://console.aws.amazon.com/vpc/.
2. In the navigation pane, choose **Route tables**.
3. Choose **Create route table**.
4. (Optional) For **Name**, enter a name for your route table.
5. For **VPC**, choose your VPC.
6. (Optional) To add a tag, choose **Add new tag** and enter the tag key and tag value.
7. Choose **Create route table**.

8. On the **Subnet associations** tab, choose **Edit subnet associations**.



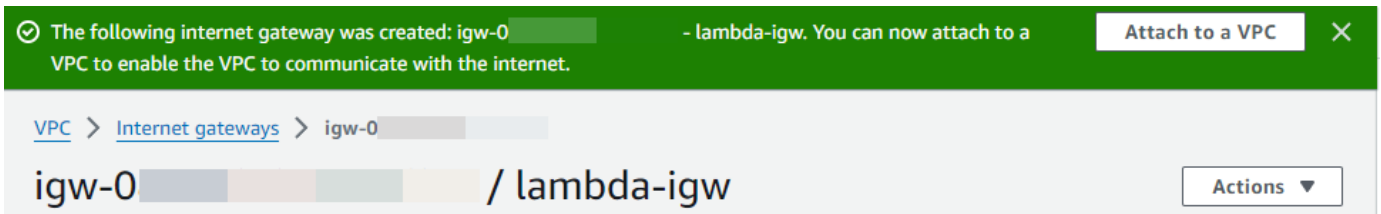
9. Select the check box for the subnet to associate with the route table.
10. Choose **Save associations**.

Create an internet gateway

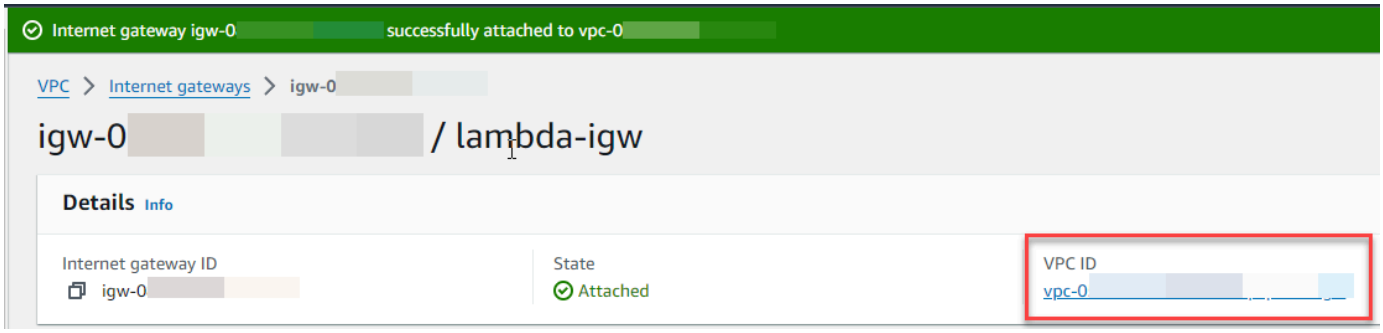
Follow these steps to create an internet gateway, attach it to your VPC, and add it to your public subnet's route table.

To create an internet gateway

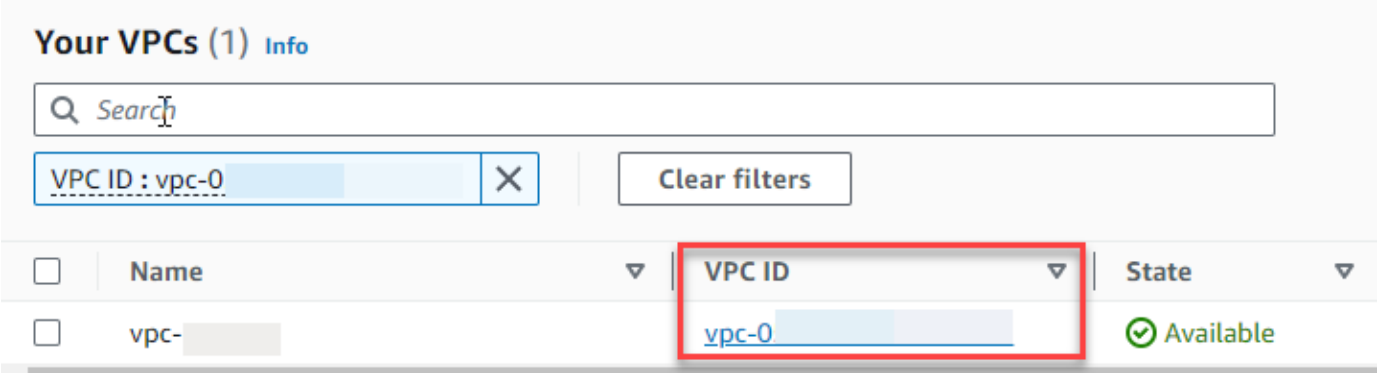
1. Open the Amazon VPC console at https://console.aws.amazon.com/vpc/.
2. In the navigation pane, choose **Internet gateways**.
3. Choose **Create internet gateway**.
4. (Optional) Enter a name for your internet gateway.
5. (Optional) To add a tag, choose **Add new tag** and enter the tag key and value.
6. Choose **Create internet gateway**.
7. Choose **Attach to a VPC** from the banner at the top of the screen, select an available VPC, and then choose **Attach internet gateway**.



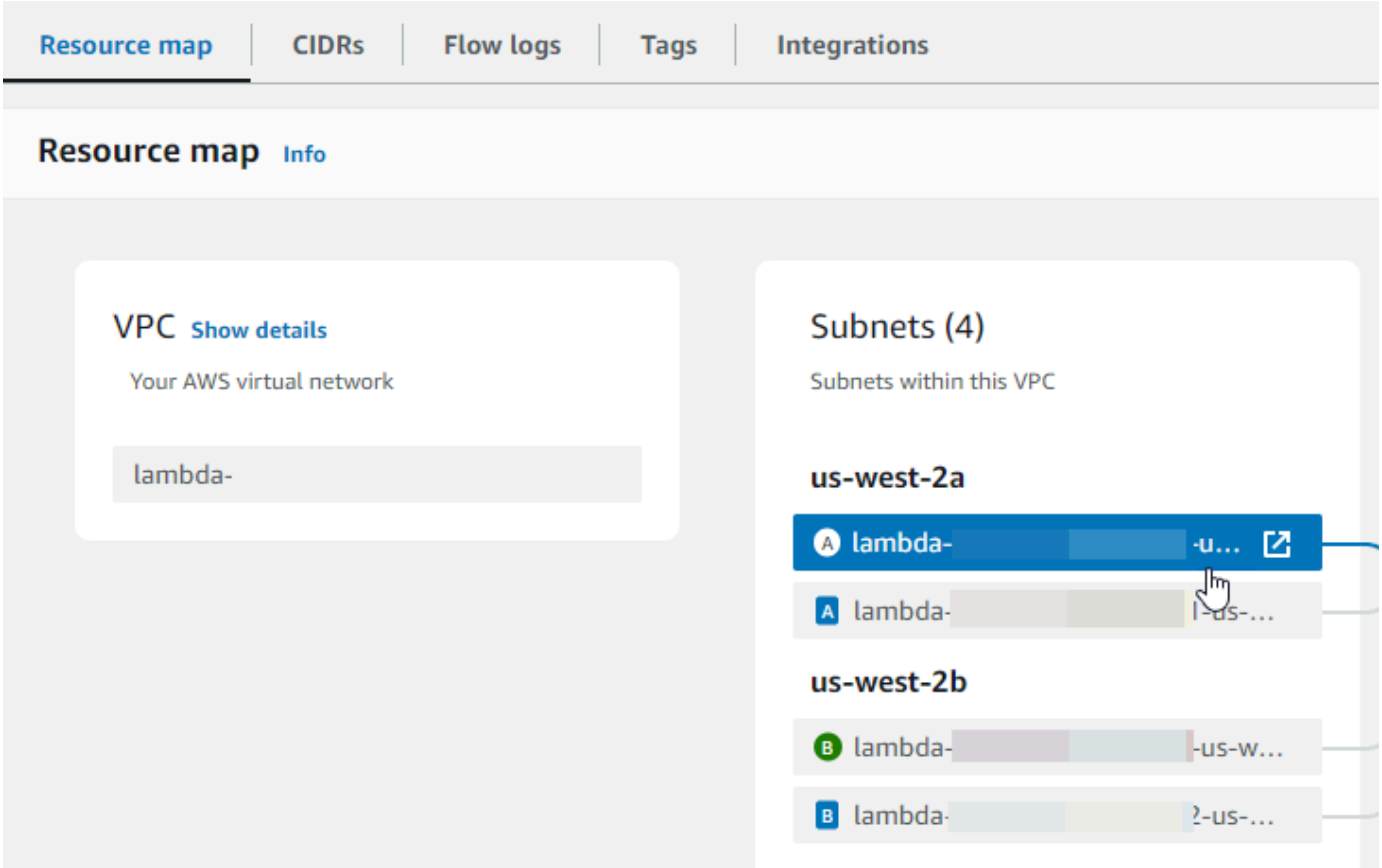
8. Choose the **VPC ID**.



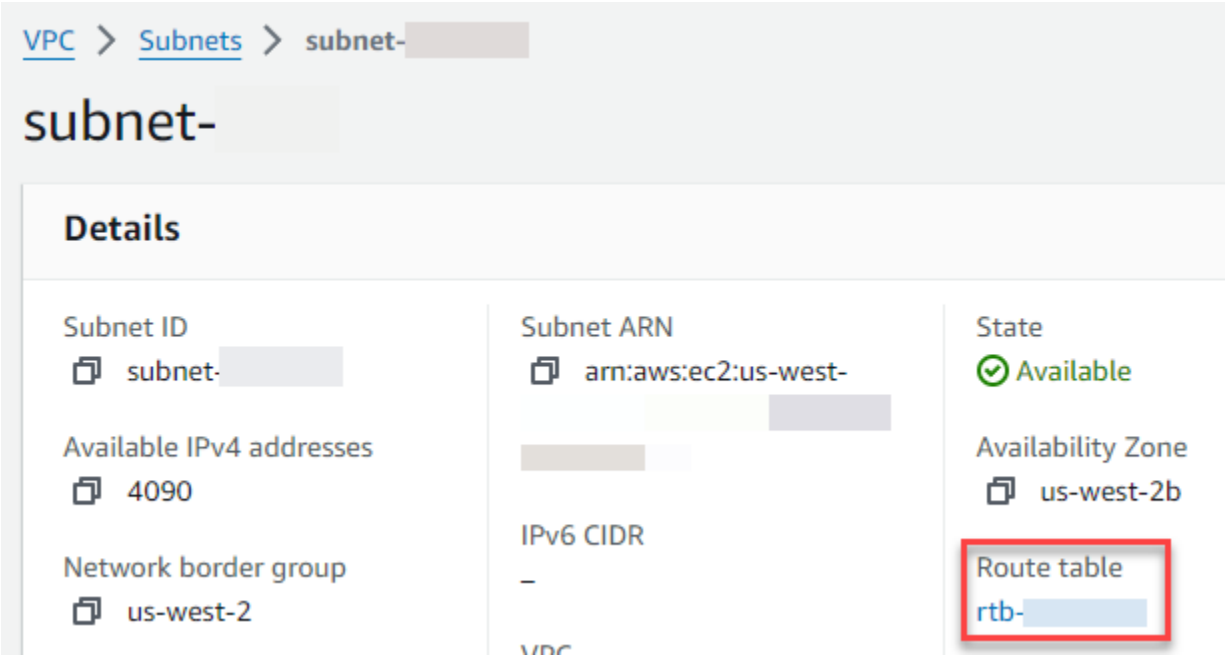
9. Choose the **VPC ID** again to open the VPC details page.



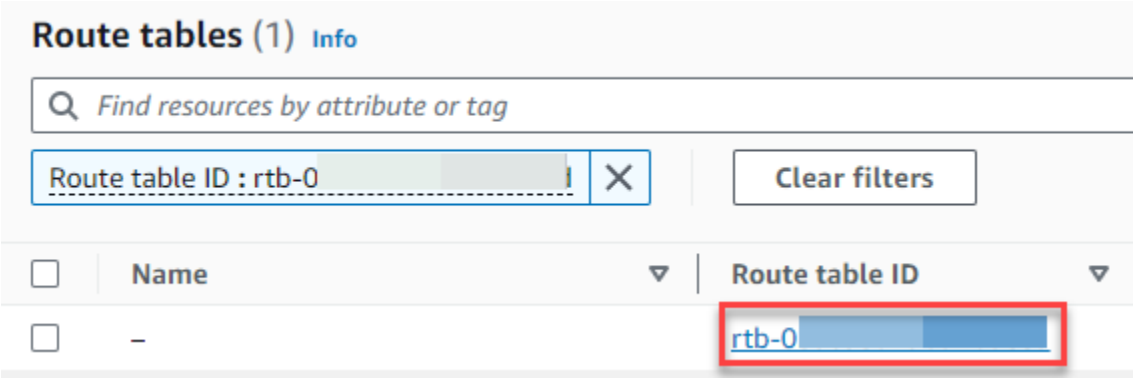
10. Scroll down to the **Resource map** section and then choose a subnet. The subnet details are displayed in a new tab.



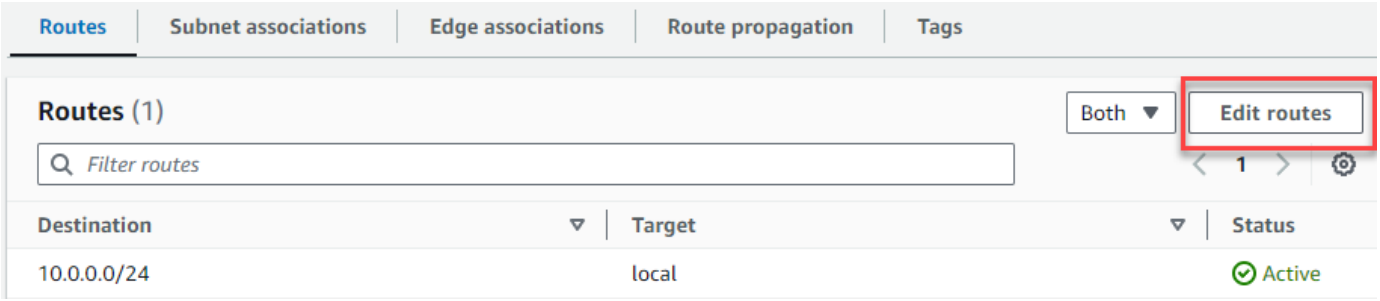
11. Choose the link under **Route table**.



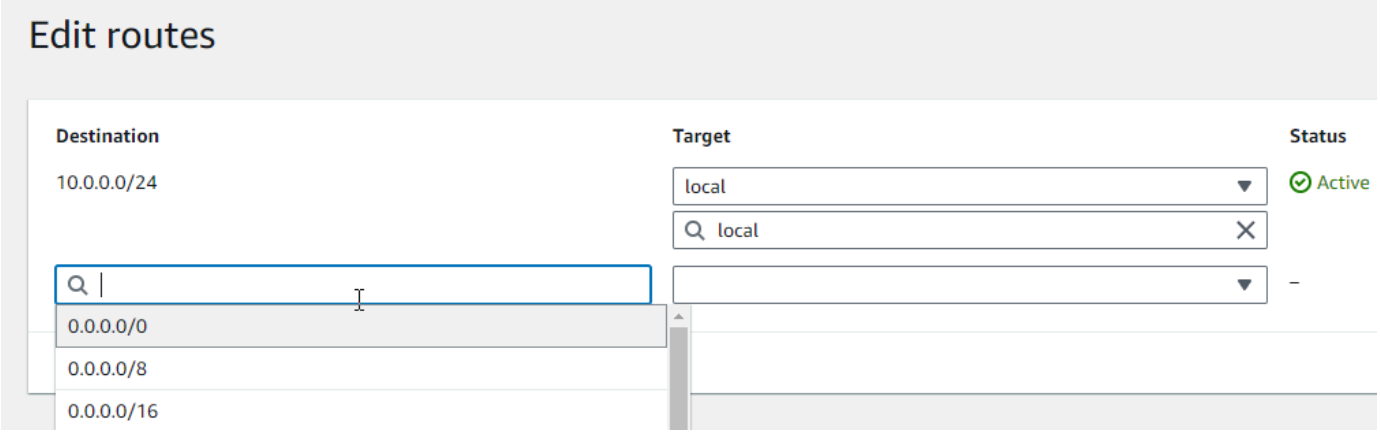
12. Choose the **Route table ID** to open the route table details page.



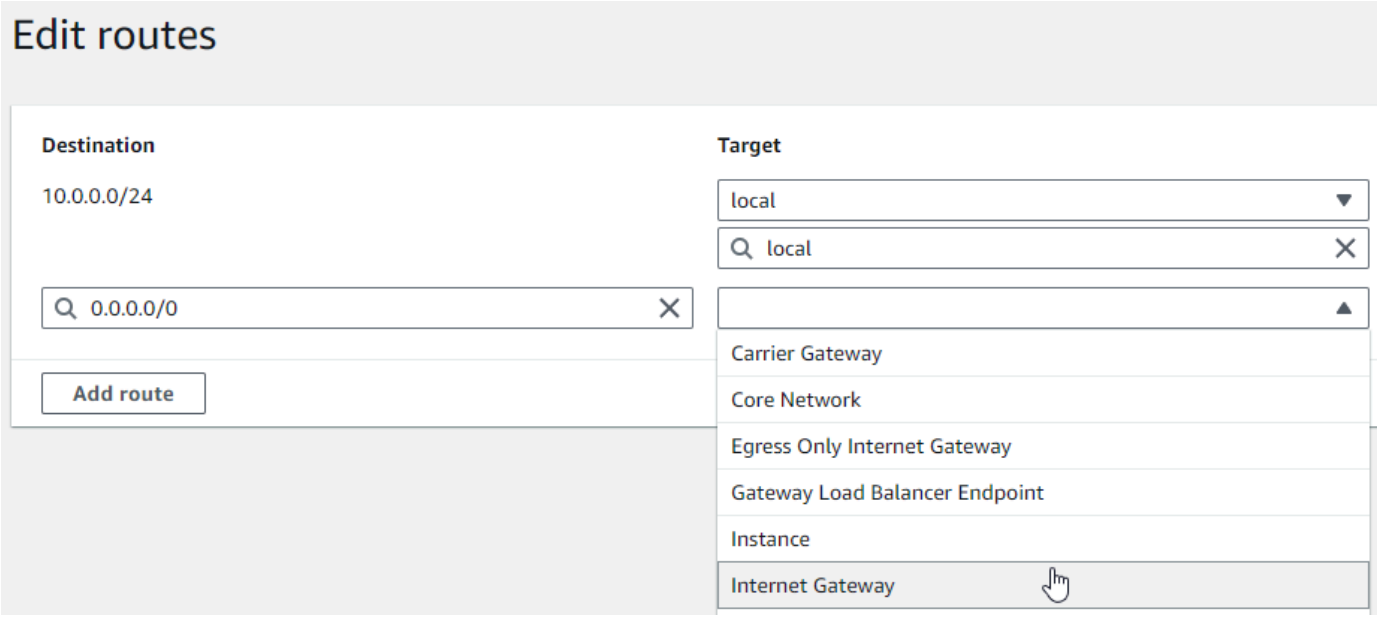
13. Under **Routes**, choose **Edit routes**.



14. Choose **Add route**, and then enter `0.0.0.0/0` in the **Destination** box.



15. For **Target**, select **Internet gateway**, and then choose the internet gateway that you created earlier. If your subnet has an IPv6 CIDR block, you must also add a route for `::/0` to the same internet gateway.



16. Choose **Save changes**.

Create a NAT gateway

Follow these steps to create a NAT gateway, associate it with a public subnet, and then add it to your private subnet's route table.

To create a NAT gateway and associate it with a public subnet

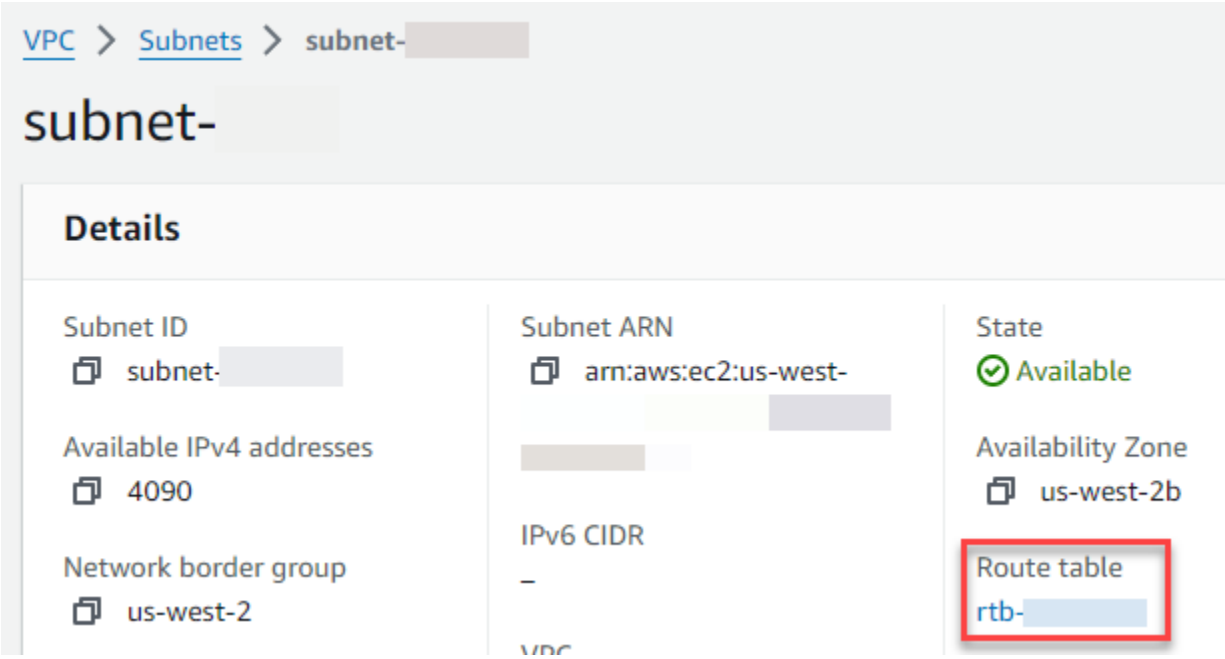
1. In the navigation pane, choose **NAT gateways**.
2. Choose **Create NAT gateway**.
3. (Optional) Enter a name for your NAT gateway.
4. For **Subnet**, select a public subnet in your VPC. (A public subnet is a subnet that has a direct route to an internet gateway in its route table.)

> **Note**
>
> NAT gateways are associated with a public subnet, but the route table entry is in the private subnet.

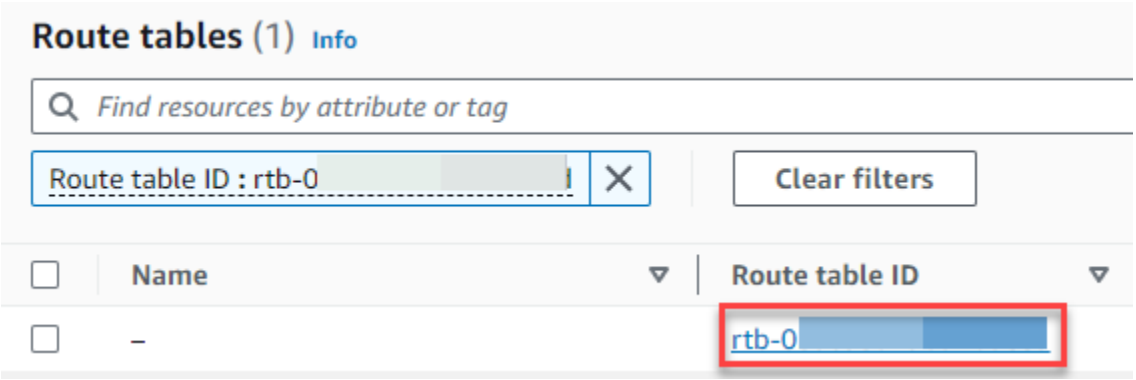
5. For **Elastic IP allocation ID**, select an elastic IP address or choose **Allocate Elastic IP**.
6. Choose **Create NAT gateway**.

To add a route to the NAT gateway in the private subnet's route table

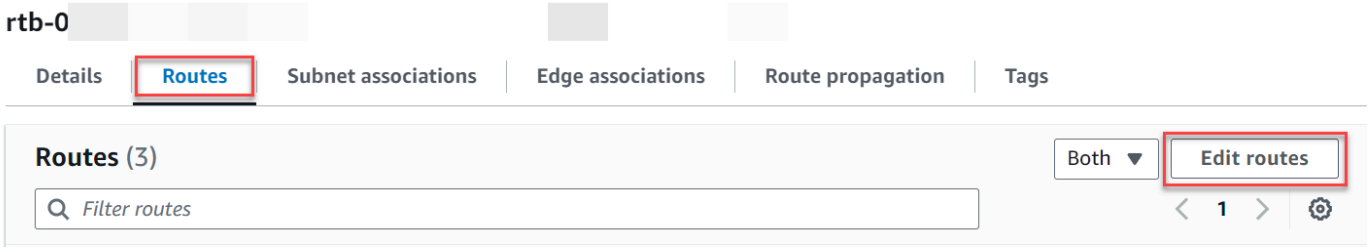
1. In the navigation pane, choose **Subnets**.
2. Select a private subnet in your VPC. (A private subnet is a subnet that doesn't have a route to an internet gateway in its route table.)
3. Choose the link under **Route table**.



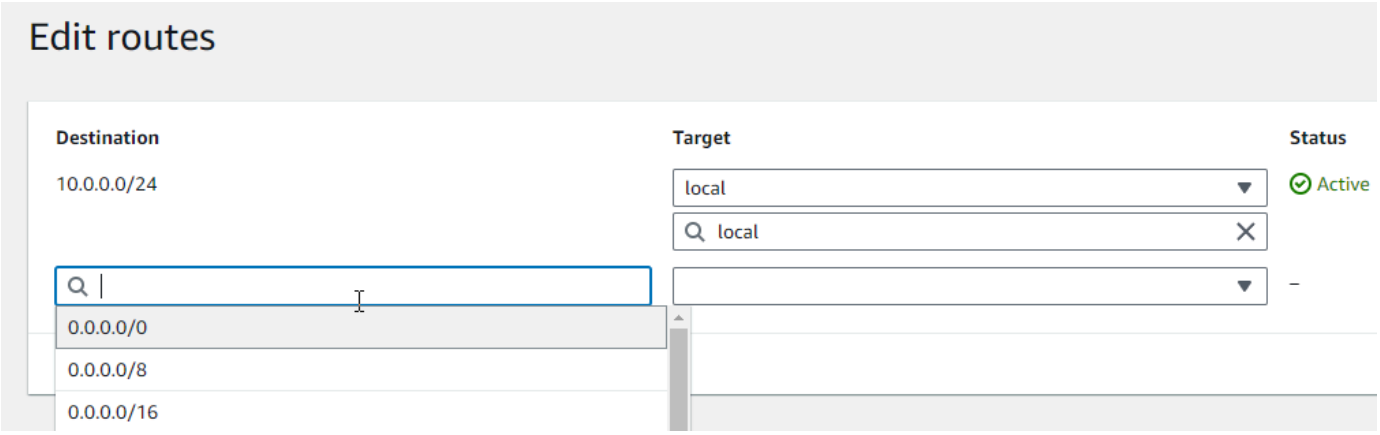
4. Choose the **Route table ID** to open the route table details page.



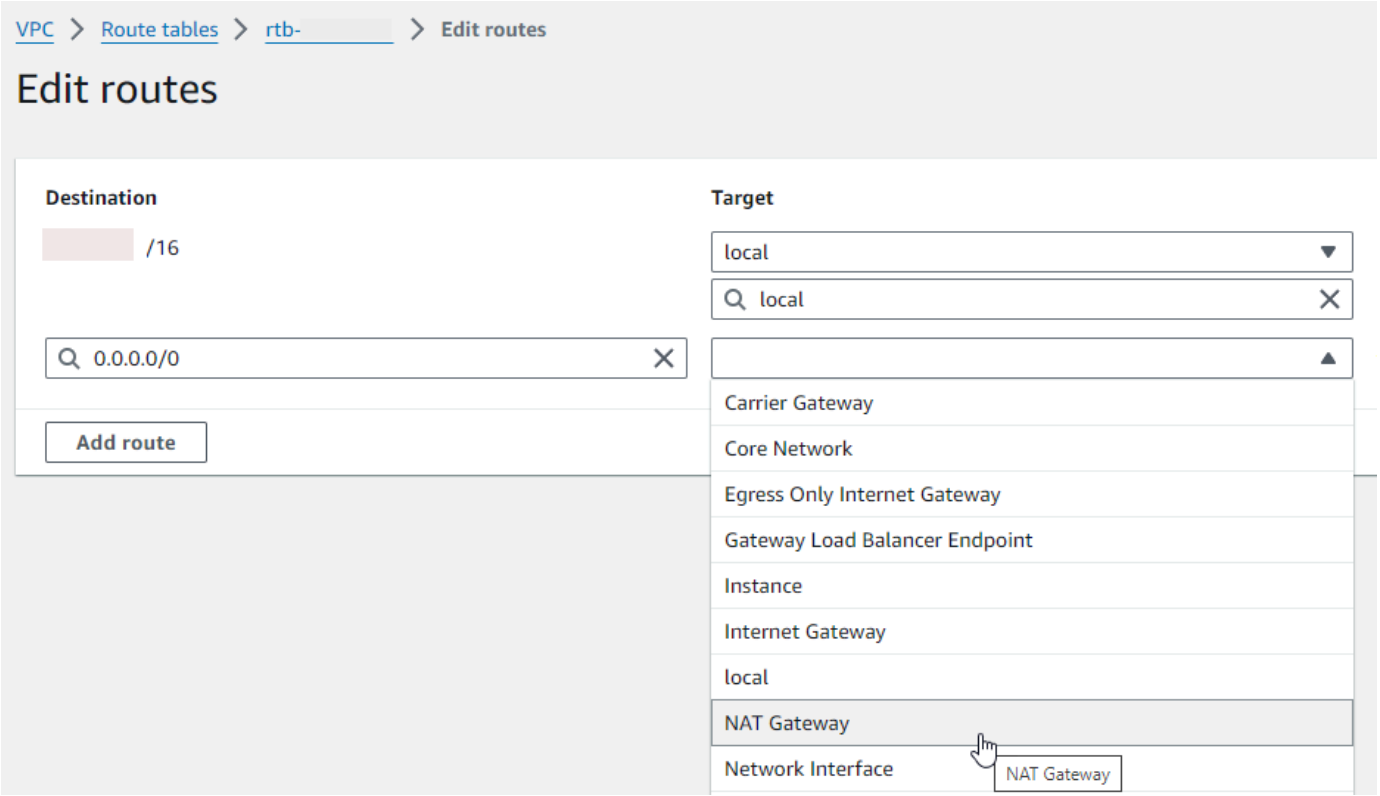
5. Scroll down and choose the **Routes** tab, then choose **Edit routes**



6. Choose **Add route**, and then enter `0.0.0.0/0` in the **Destination** box.



7. For **Target**, select **NAT gateway**, and then choose the NAT gateway that you created earlier.



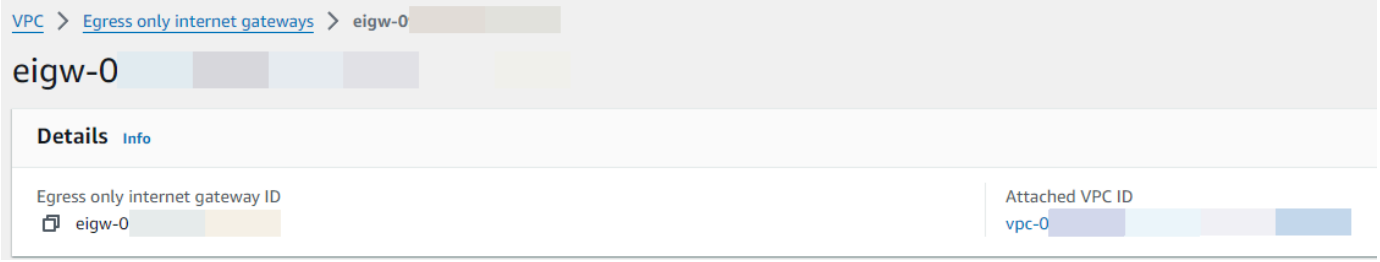
8. Choose **Save changes**.

Create an egress-only internet gateway (IPv6 only)

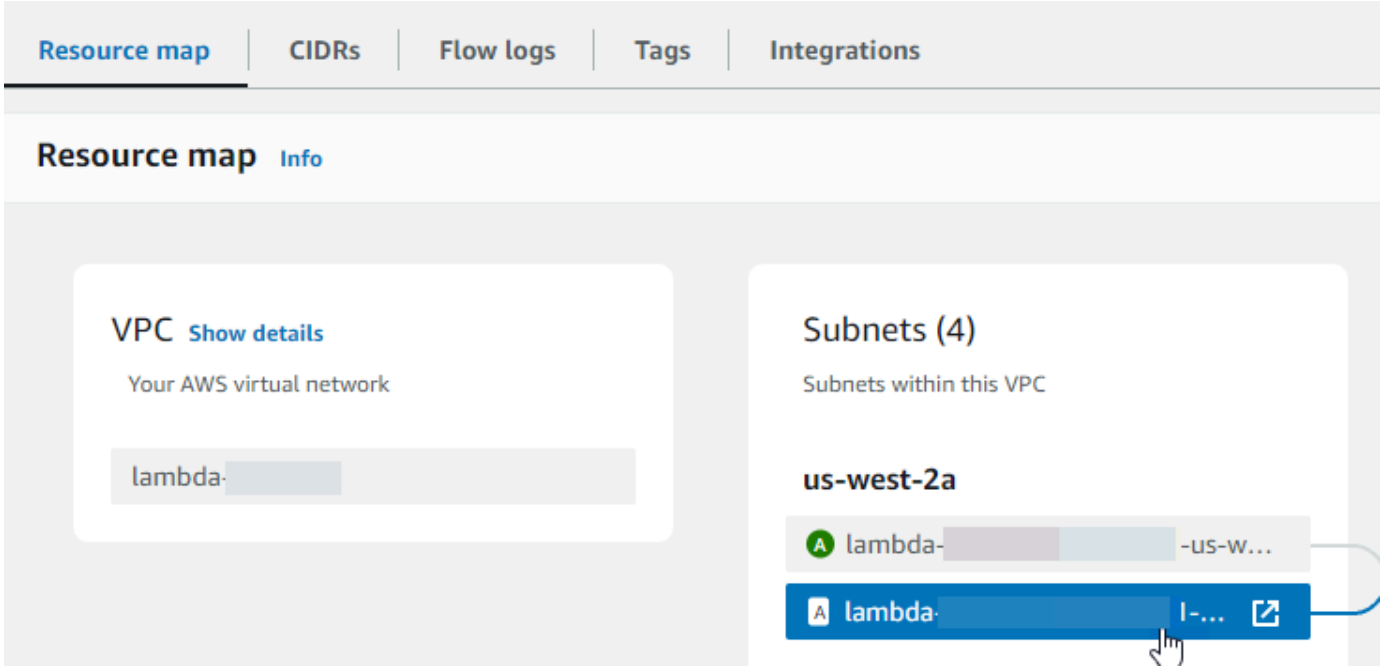
Follow these steps to create an egress-only internet gateway and add it to your private subnet's route table.

To create an egress-only internet gateway

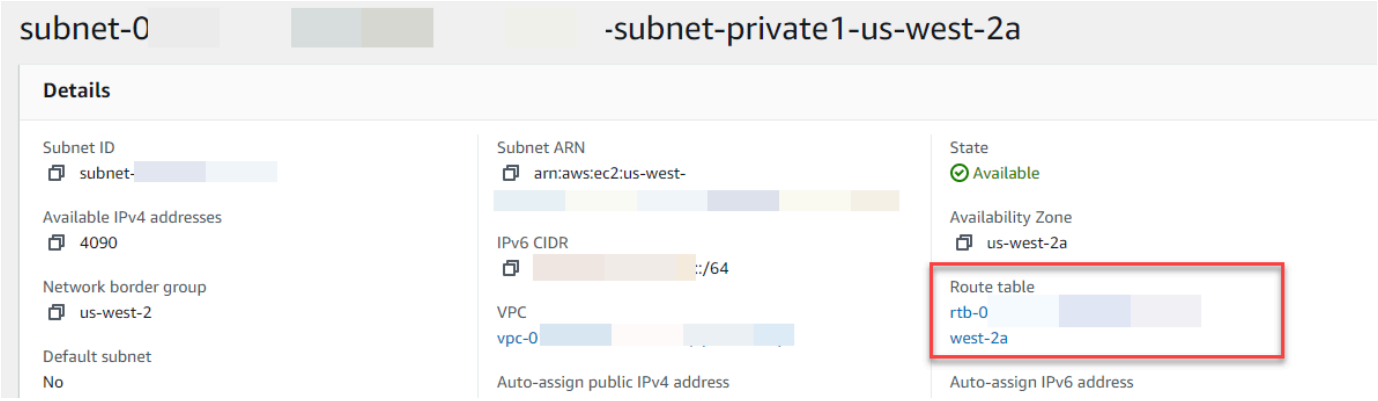
1. In the navigation pane, choose **Egress-only internet gateways**.
2. Choose **Create egress only internet gateway**.
3. (Optional) Enter a name.
4. Select the VPC in which to create the egress-only internet gateway.
5. Choose **Create egress only internet gateway**.
6. Choose the link under **Attached VPC ID**.



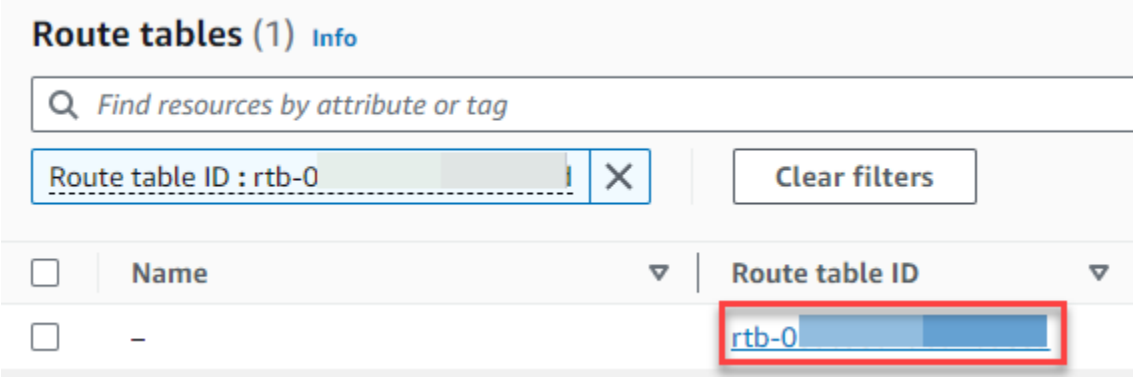
7. Choose the link under **VPC ID** to open the VPC details page.
8. Scroll down to the **Resource map** section and then choose a private subnet. (A private subnet is a subnet that doesn't have a route to an internet gateway in its route table.) The subnet details are displayed in a new tab.



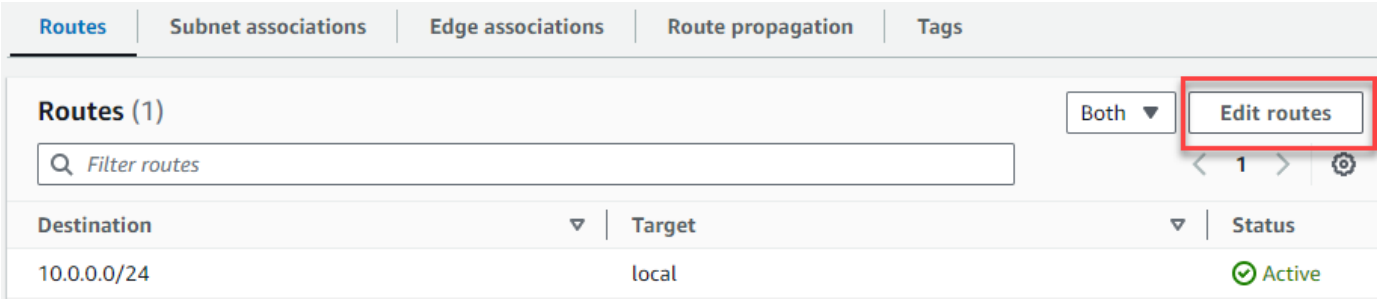
9. Choose the link under **Route table**.



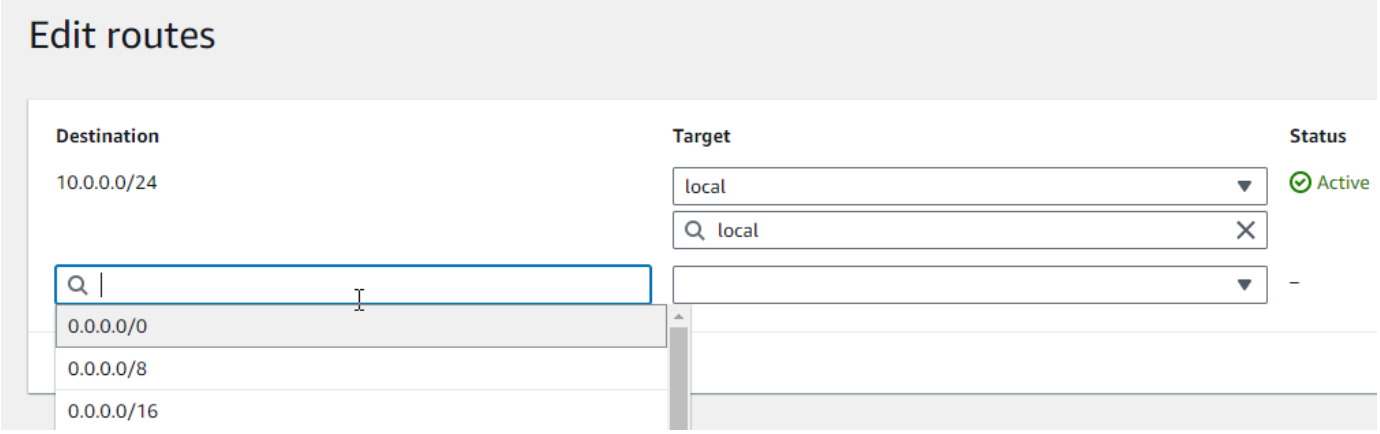
10. Choose the **Route table ID** to open the route table details page.



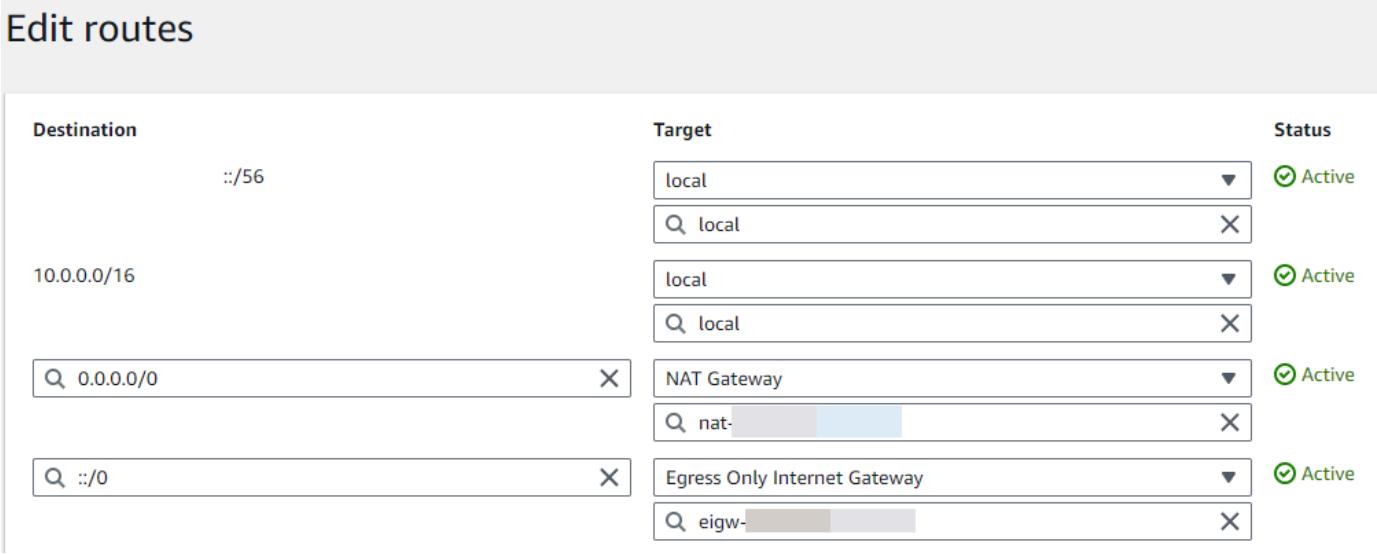
11. Under **Routes**, choose **Edit routes**.



12. Choose **Add route**, and then enter `::/0` in the **Destination** box.



13. For **Target**, select **Egress Only Internet Gateway**, and then choose the gateway that you created earlier.



14. Choose **Save changes**.

Configure the Lambda function

To configure a VPC when you create a function

1. Open the Functions page of the Lambda console.
2. Choose **Create function**.
3. Under **Basic information**, for **Function name**, enter a name for your function.
4. Expand **Advanced settings**.
5. Select **Enable VPC**, and then choose a VPC.
6. (Optional) To allow outbound IPv6 traffic, select **Allow IPv6 traffic for dual-stack subnets**.
7. For **Subnets**, select all private subnets. The private subnets can access the internet through the NAT gateway. Connecting a function to a public subnet doesn't give it internet access.

> **Note**
>
> If you selected **Allow IPv6 traffic for dual-stack subnets**, all selected subnets must have an IPv4 CIDR block and an IPv6 CIDR block.

8. For **Security groups**, select a security group that allows outbound traffic.
9. Choose **Create function**.

Lambda automatically creates an execution role with the AWSLambdaVPCAccessExecutionRole AWS managed policy. The permissions in this policy are required only to create elastic network interfaces for the VPC configuration, not to invoke your function. To apply least-privilege permissions, you can remove the **AWSLambdaVPCAccessExecutionRole** policy from your execution role after you create the function and VPC configuration. For more information, see Required IAM permissions.

To configure a VPC for an existing function

To add a VPC configuration to an existing function, the function's execution role must have permission to create and manage elastic network interfaces. The AWSLambdaVPCAccessExecutionRole AWS managed policy includes the required permissions. To apply least-privilege permissions, you can remove the **AWSLambdaVPCAccessExecutionRole** policy from your execution role after you create the VPC configuration.

1. Open the Functions page of the Lambda console.
2. Choose a function.
3. Choose the **Configuration** tab, and then choose **VPC**.
4. Under **VPC**, choose **Edit**.
5. Select the VPC.
6. (Optional) To allow outbound IPv6 traffic, select **Allow IPv6 traffic for dual-stack subnets**.
7. For **Subnets**, select all private subnets. The private subnets can access the internet through the NAT gateway. Connecting a function to a public subnet doesn't give it internet access.

   > **Note**
   >
   > If you selected **Allow IPv6 traffic for dual-stack subnets**, all selected subnets must have an IPv4 CIDR block and an IPv6 CIDR block.

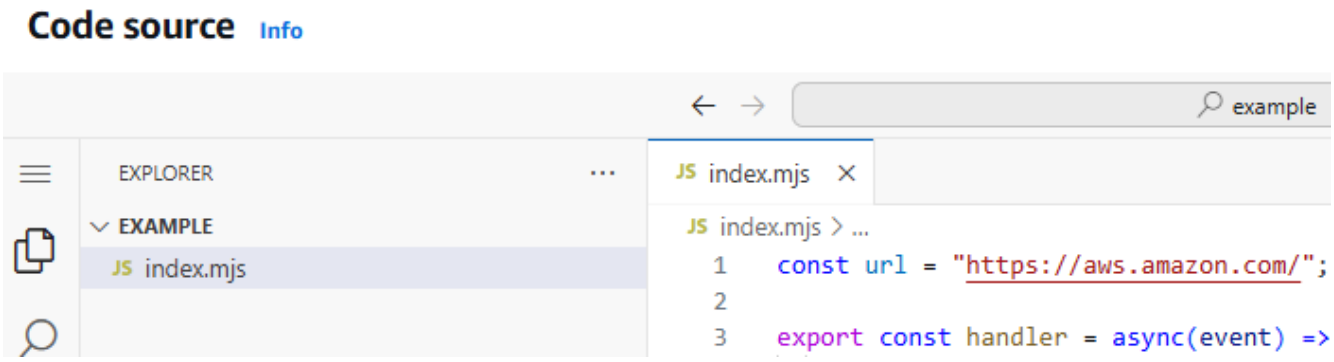
8. For **Security groups**, select a security group that allows outbound traffic.
9. Choose **Save**.

Test the function

Use the following sample code to confirm that your VPC-connected function can reach the public internet. If successful, the code returns a `200` status code. If unsuccessful, the function times out.

Node.js

1. In the **Code source** pane on the Lambda console, paste the following code into the **index.mjs** file. The function makes an HTTP GET request to a public endpoint and returns the HTTP response code to test if the function has access to the public internet.

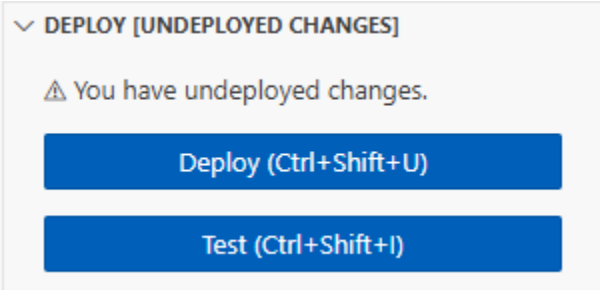


Example— HTTP request with async/await

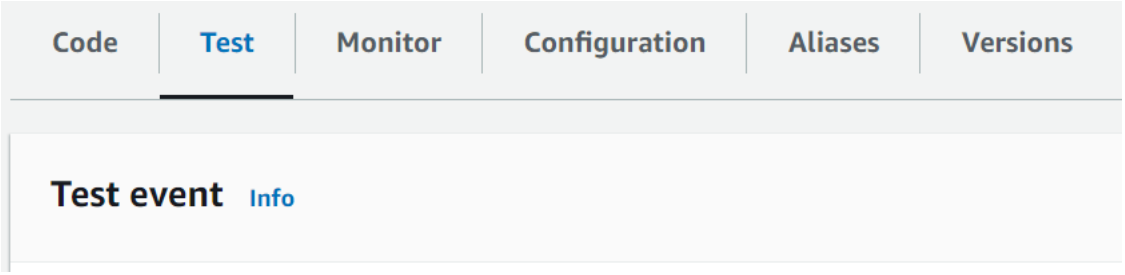
```
const url = "https://aws.amazon.com/";

export const handler = async(event) => {
    try {
        const res = await fetch(url);
        console.info("status", res.status);
        return res.status;
    }
    catch (e) {
        console.error(e);
        return 500;
    }
};
```

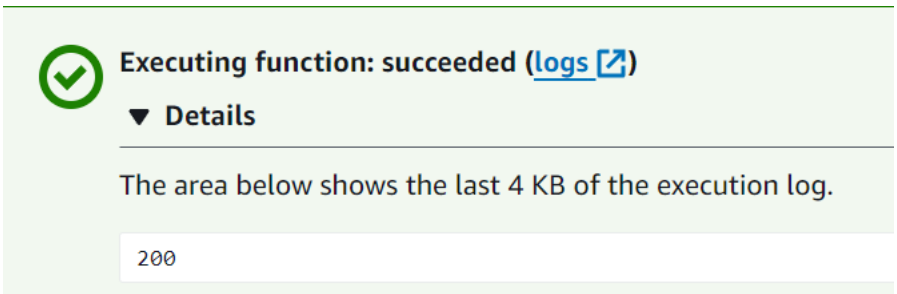
2. In the **DEPLOY** section, choose **Deploy** to update your function's code:



3. Choose the **Test** tab.



4. Choose **Test**.
5. The function returns a `200` status code. This means that the function has outbound internet access.

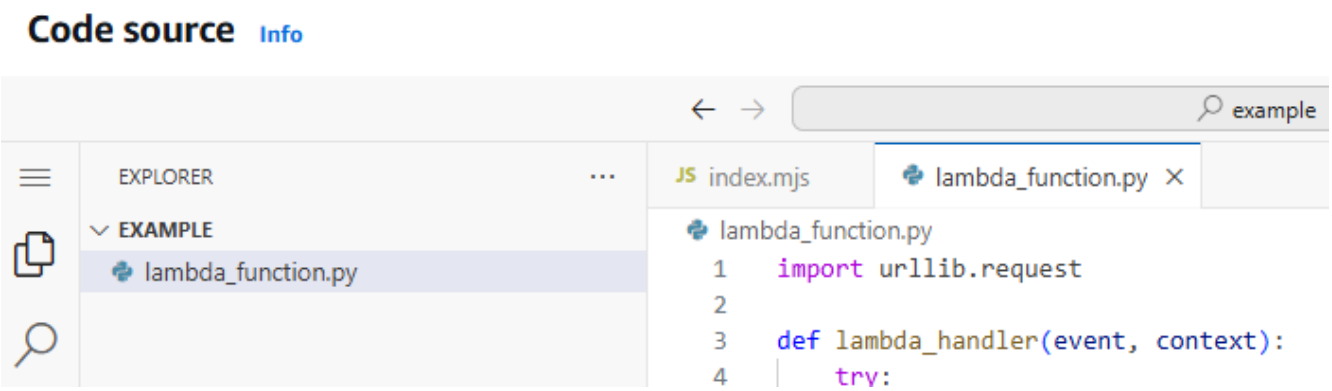


If the function can't reach the public internet, you get an error message like this:

```
{
  "errorMessage": "2024-04-11T17:22:20.857Z abe12jlc-640a-8157-0249-9be825c2y110
 Task timed out after 3.01 seconds"
}
```

Python

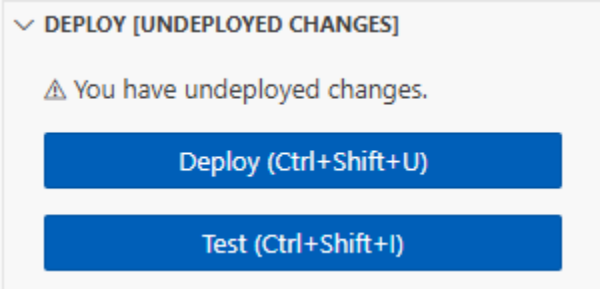
1. In the **Code source** pane on the Lambda console, paste the following code into the **lambda_function.py** file. The function makes an HTTP GET request to a public endpoint and returns the HTTP response code to test if the function has access to the public internet.



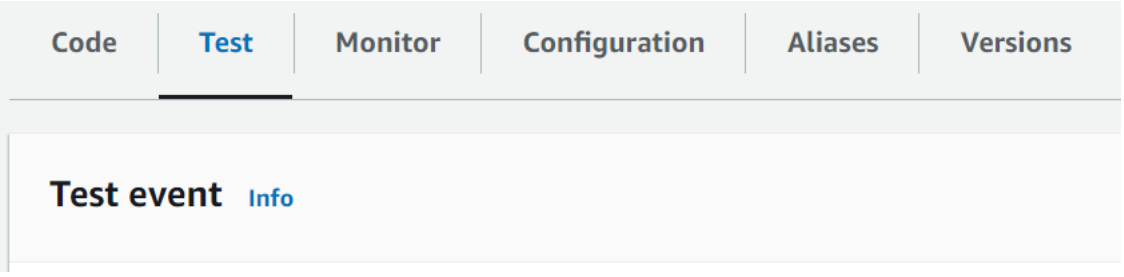

```
import urllib.request

def lambda_handler(event, context):
    try:
        response = urllib.request.urlopen('https://aws.amazon.com')
        status_code = response.getcode()
        print('Response Code:', status_code)
        return status_code
    except Exception as e:
        print('Error:', e)
        raise e
```

2. In the **DEPLOY** section, choose **Deploy** to update your function's code:

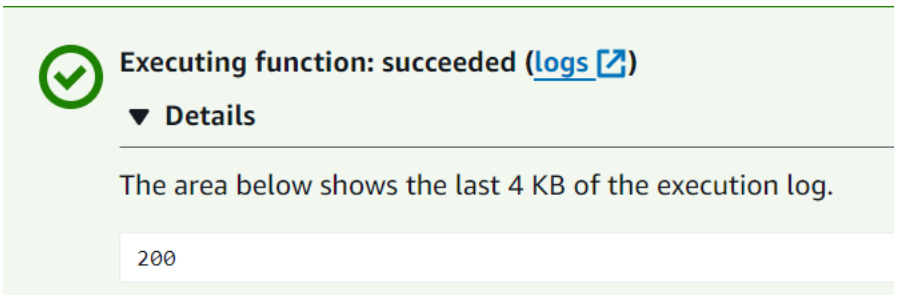


3. Choose the **Test** tab.



4. Choose **Test**.

5. The function returns a 200 status code. This means that the function has outbound internet access.



If the function can't reach the public internet, you get an error message like this:

```
{
  "errorMessage": "2024-04-11T17:22:20.857Z abe12jlc-640a-8157-0249-9be825c2y110
 Task timed out after 3.01 seconds"
}
```